

## Jon Pulsipher – CS6015 – A4: Malloc Replacement – 3/24/26

Here is a timing comparison of MyMalloc and C++'s built-in malloc:

For 100000 small values:

MyMalloc: 745ms

malloc: 1ms

For 100000 big values of size 10000:

MyMalloc: 732ms

malloc: 64ms

For 100000 mixed values of size 10000 and ints:

MyMalloc: 749ms

malloc: 30ms

For 1 int:

MyMalloc: 8 microseconds

malloc: 0 microseconds

The built-in malloc is very fast, needing only ~1 ms to allocate 100,000 ints, and less than 0.5 microseconds to allocate a single int. Its runtime only reaches into the millisecond range when it is allocating very-large values, and the runtime increase seems proportional to how many large variables are allocated (the mixed test took half the time of the big test and allocated half as many large variables).

In comparison, MyMalloc requires almost a full second to allocate 100,000 ints, and that long runtime is the same for large, small, or mixed timing tests. Its runtime for the “minimum” test of 1 int is 8 microseconds, as opposed to malloc's much smaller runtime.

What accounts for these differences?

With MyMalloc's current implementation, any allocation calls mmap and requests at least a full page. The test-size for this timing test was 10,000, so a pointer to 3 pages would need to be provided, but even ints will request a full page. This is a relatively slow syscall, is not how malloc does it, and is wasting space that could be utilized. If un-utilized portions of a page that had already been mmap'ed could be returned on a call to allocate, this would speed up significantly. In class, we discussed a strategy of pre-allocating a page and reserving portions of it for small (int), medium (double), and large data (remaining bytes, potentially multiple pages). Pursuing that strategy would significantly increase the performance of MyMalloc.

The slowdown in MyMalloc's startup time may be contributed to by the necessity to initialize its HashTable for any use, including small use. The built-in malloc may have optimizations that allow it to set up less if it is utilized less, or have its set up done in the background. MyMalloc isn't integrated into C/C++ in the way that the built-in malloc is, and may not be able to reach the kind of efficiency malloc has that allows it to malloc/free an int in less than half a microsecond.

*How to run timing and unit tests:*

Timing: MallocReplacement.exe

Tests: MyMallocTests.exe, HashTableTests.exe